

Errores comunes segundo parcial poo

Juan Ignacio Raggio

June 4, 2025

Contents

1	Generales	1
1.1	<code><=>(other)</code> vs <code>==(other)</code> vs <code>equals(other)</code> - Comparable . .	2
1.1.1	Conclusion de pasaje de no instancia:	2
2	Java	2
2.1	Prestarle mucha atencion a la coleccion que se devuelve . . .	2
3	Ruby	2
3.1	Leer bien los test y <u>Los codigos que te pueden dar</u> para que sean parte de tu implementacion	2
3.2	<code>to_s</code>	3
3.3	Metodos <code>protected</code>	3
3.4	Clases abstractas	3
3.4.1	DONE DOUBT Aca siempre me tira warnings el ide, entiendo porque es, no entiendo entonces porque estamos forzando a hacerlo de esta manera	4
3.4.2	DONE DOUBT En el ejercicio de Accounts del tp8 no se porque crean un metodo de la siguiente forma: . . .	4
3.5	Colecciones	4
3.5.1	No olvidarse de sobrescribir <code>==(other)</code> y <code>hash</code>	5

1 Generales

- No olvidarse de implementar metodos `toString`, `to_s`, etc.
- Leer muy detenidamente el test y no olvidarse metodos que pidan implementar

1.1 $\leq$ (other) vs ==(other) vs equals(other) - Comparable

- Ojo porque en caso de ruby si a $\leq$ se le pasa un objeto que no es instancia de self, retorna: **nil**
- Si a ==(other) o a equals(other) le pasas una no-instancia-self **false**

1.1.1 Conclusion de pasaje de no instancia:

Tipo de metodo	Ruby	Java
Comparacion	nil	No compila
Equidad	false	false

2 Java

2.1 Prestarle mucha atencion a la coleccion que se devuelve

Es la parte facil pero al hacerla rapido se puede meter la pata

- Retornar una Coleccion / Mapa que cumpla ciertas condiciones
- Acordarse que

$$Map \neq Collection \wedge Map \not\subseteq Iterable$$

- Por lo tanto es muy importante que si tengo que devolver un mapa, devuelvo un mapa, niquiera un Iterable porque no es Iterable un Mapa:

3 Ruby

3.1 Leer bien los test y Los codigos que te pueden dar para que sean parte de tu implementacion

- A veces aparecen clases que tenes que implementar y no son tan explicitas (como en el de passHolder)
- El metodo map puede ser muy util:

```
class LimitedDaysPassholders < ThemeParkPassholders
```

```

def initialize(visitor, days_cap)
  super(visitor)
  @days_cap = days_cap
end

def add_pass?(park, day)
  # Donde @passes es un SortedSet
  # y day es un atributo de los objetos que se guardan @passes
  super && @passes.length < @days_cap &&
    !@passes.map{ |pass| pass.day }.include?(day)
end
end

```

3.2 to_s

- No olvidarse de hacer los to_s
- Es mejor hacer interpolacion de strings que concatenacion

```

i = "interpolacion"
"Esto es una #{i} de strings"

```

3.3 Metodos protected

- En caso de que un metodo necesite leer una variable de instancia de otra instancia de la misma clase: Es **MUY IMPORTANTE** hacer que esos getters sean **protected** porque sino estariamos leakeando variables sin sentido (ojo, puede que en algun caso se quiera dejar normal)

3.4 Clases abstractas

Sabemos de objetos que las clases abstractas **NUNCA** pueden instanciarse, por lo que el metodo **initialize** de ruby en una “clase abstracta” a pesar de que no existan formalmente, los vamos a simular haciendo que el initialize tenga siempre la siguiente forma:

```

class Pirlas
  def initialize
    raise 'Este metodo debe sobreescribirse'
  end
end

```

end

Otros metodos del pirlas
end

3.4.1 DONE DOUBT Aca siempre me tira warnings el ide, entiendo porque es, no entiendo entonces porque estamos forzando a hacerlo de esta manera

Si, no importan los warnings para clases abstractas

3.4.2 DONE DOUBT En el ejercicio de Accounts del tp8 no se porque crean un metodo de la siguiente forma:

```
def extract?(_amount)
  raise '...'
end
```

- ☒ Porque el `_`? y porque cuando “sobreescriben” este metodo no lo ponen? es para que no tire warnings? El `_` se usa cuando ese parametro no lo vas a usar dentro del metodo
- ☒ En el initialize igualmente seguiriamos con el mismo problema Si, no importan los warnings para clases abstractas

3.5 Colecciones

Muy importante:

- Es muy comun cuando quiero extraer info de las colecciones, ya sea: Hash, Set, etc. Tener que **pasarlas a array**, por ejemplo: En este caso, tenemos `@bag` que es un Hash table y cuando queremos obtener las llaves para ver si estamos en el final, usamos `.keys` para obtenerlas

```
def to_s
  keys = @bag.keys
  s = ''
  keys.each {
    s += "#{it.to_s} => #{@bag[it].to_s}"
    s += ", " unless it == keys.last
  }
  "#{s}"
end
```

3.5.1 No olvidarse de sobrescribir ==(**other**) y hash

Retornan false si other no es instancia

- **OJO** porque cuando solo se sobrescribe al ==(**other**) NO OLVIDARSE DEL ALIAS Como las colecciones usan eql? para comparar, si no lo sobrescribimos no funciona

```
alias_method :eql?, :==
```